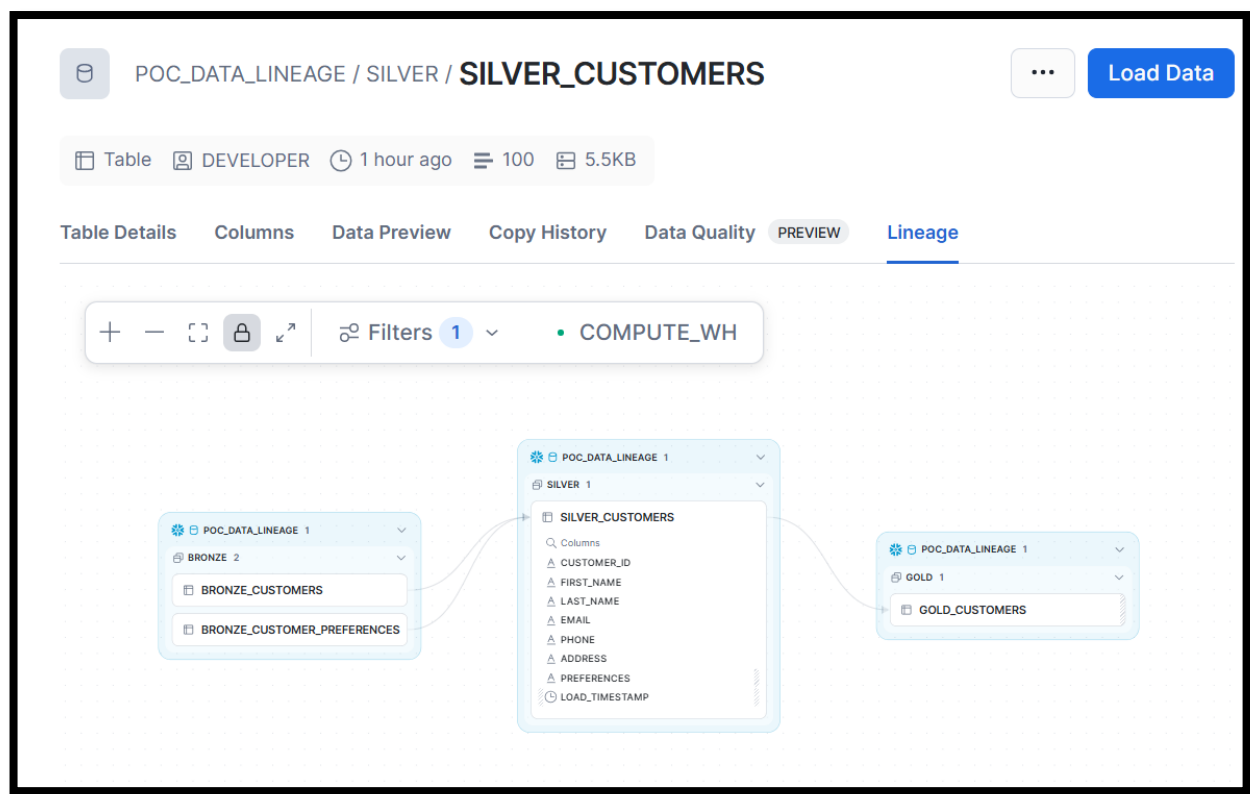


Data Lineage Scenarios - Summary

Scenario	Description	Internal/External	Lineage Generated
S1	Load Silver table with join from 2 Bronze tables	Internal	Table + Column level
S2	Load Silver table with join via configuration table	Internal	Table + Column level
S3	Data load by JavaScript stored procedure	Internal	Table + Column level
S4	Airflow external lineage for Customer table	External	Table Level
S5	Openflow external lineage for Customer table	External	Table + Column level
S6	Data-level lineage tracking for transformed columns using CoCo	Internal	Column level (with transformation logic)
S7	External Orchestration Lineage Gap (ADF + Nexla)	-	-
S8	Data load by Snowpark stored procedure	Internal	Table + Column level
S9	Use Python library for calculations	Internal	Table + Column level
S10	Use Python stored procedure to update column	Internal	Column level (logic not shown)
S11	Rename column in Silver table	Internal	Column level
S12	Delete column from Bronze table	Internal	Column level
S13	Drop Table	Internal	Table Level
S14	GET_LINEAGE() function	-	-

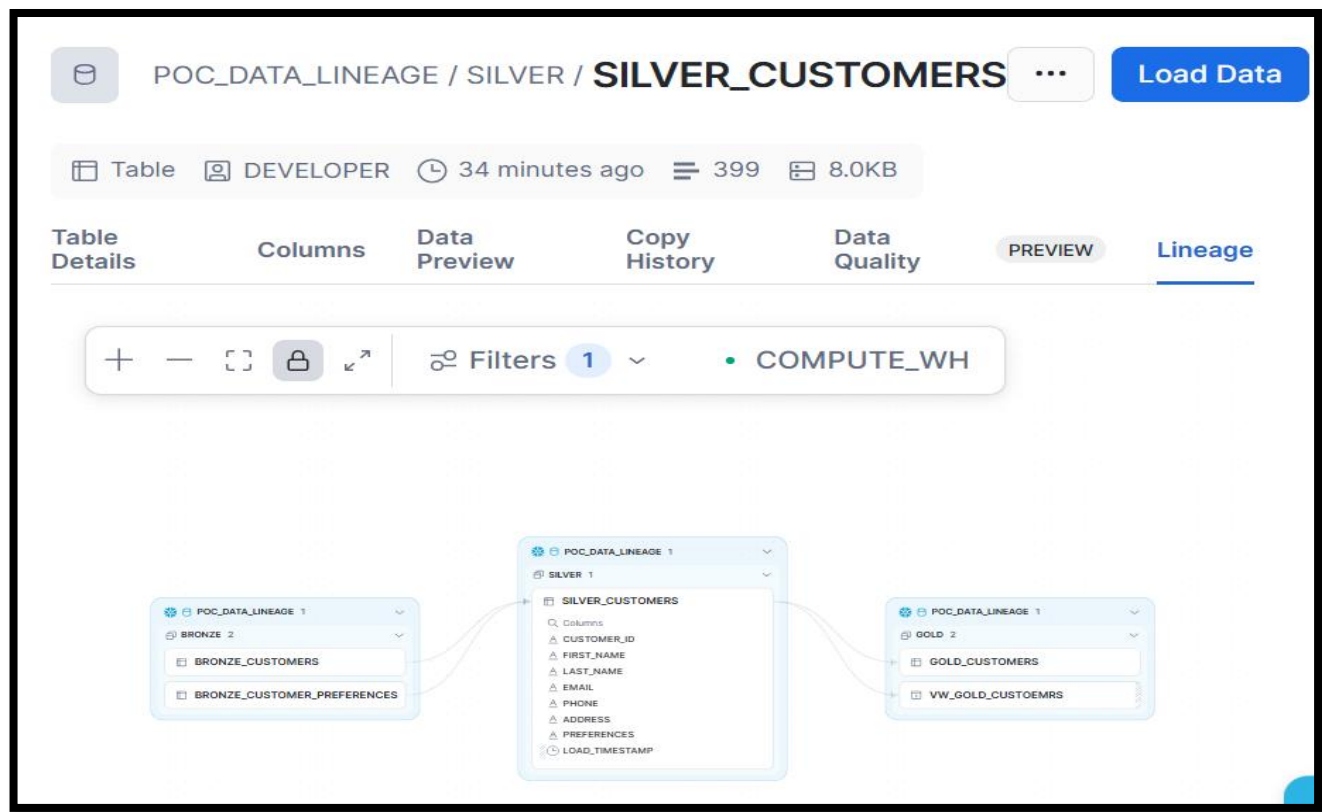
Scenario 1: Load 1 silver table with a join from 2 bronze tables .



Scenario 2: Load 1 silver table with a join from 2 bronze tables using configuration tables.

Description: Table SILVER.SILVER_GOLD loaded by joining two BRONZE tables - BRONZE.BRONZE_CUSTOMERS and BRONZE.BRONZE_CUSTOMER_PREFERENCES by stored procedure SP_LOAD_FROM_CONFIG_JOIN using configuration table.

Observation 1: Table level lineage successfully captured. Both upstream and downstream flows captured.



Observation 2: The stored procedure name and query used to load the SILVER.SILVER_CUSTOMERS table is seen.

POC_DATA_LINEAGE / SILVER / SILVER_CUSTOMERS

Load Data

Table

DEVELOPER

36 minutes ago

399

8.0KB

Table Details

Columns

Data Preview

Copy History

Data Quality

PREVIEW

Lineage

POC_DATA_LINEAGE 1

BRONZE 2

BRONZE_CUSTOMERS

BRONZE_CUSTOMER_PREFERENCES

BRONZE_CUSTOMERS to SILVER_CUSTO...

LEFT JOIN

POC_DATA_LINEAGE.BRONZE.BRONZE_CUSTOM

ER_PREFERENCES bcp

ON bc.customer_id =

bcp.customer_id

Stored Procedures

Direct

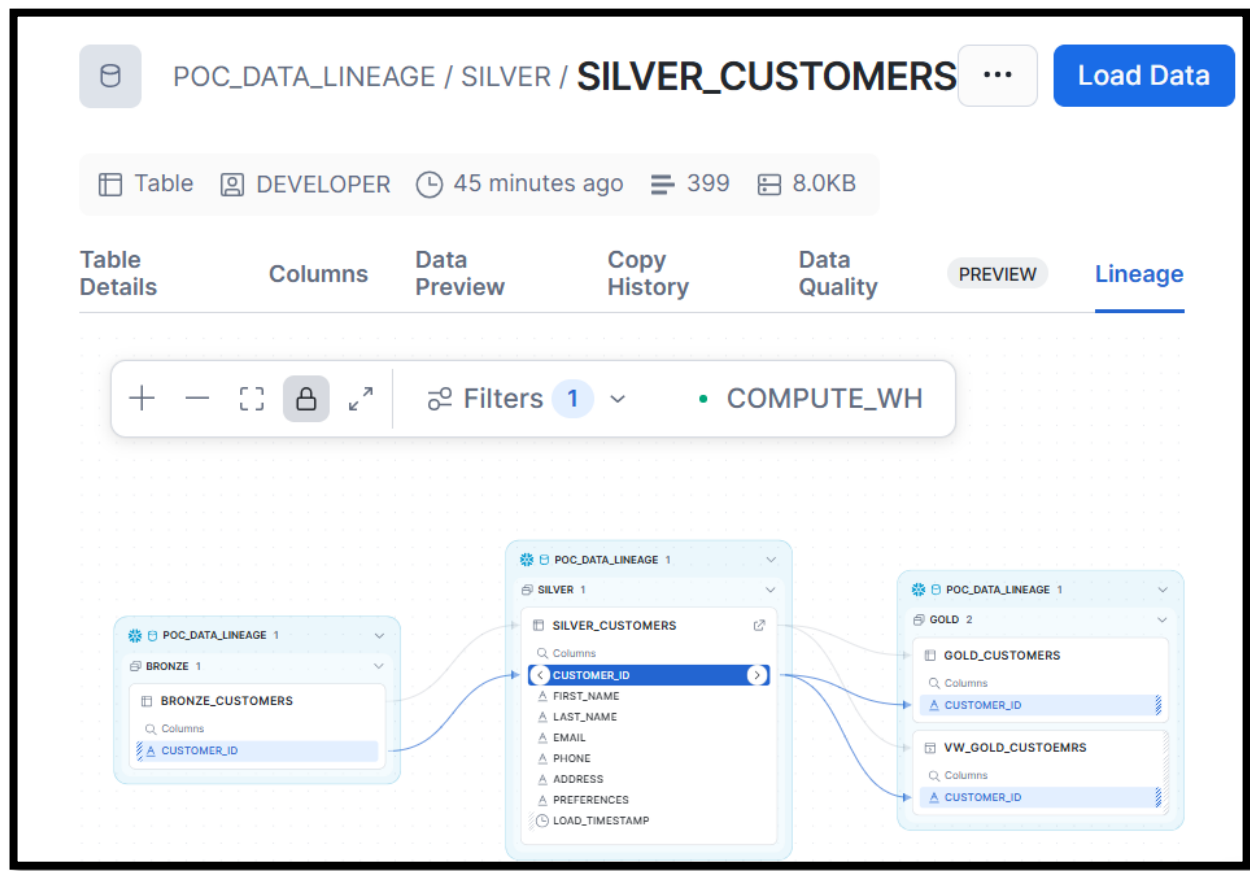
SP_LOAD_FROM_CONFIG_JOII

Root

—

Tasks

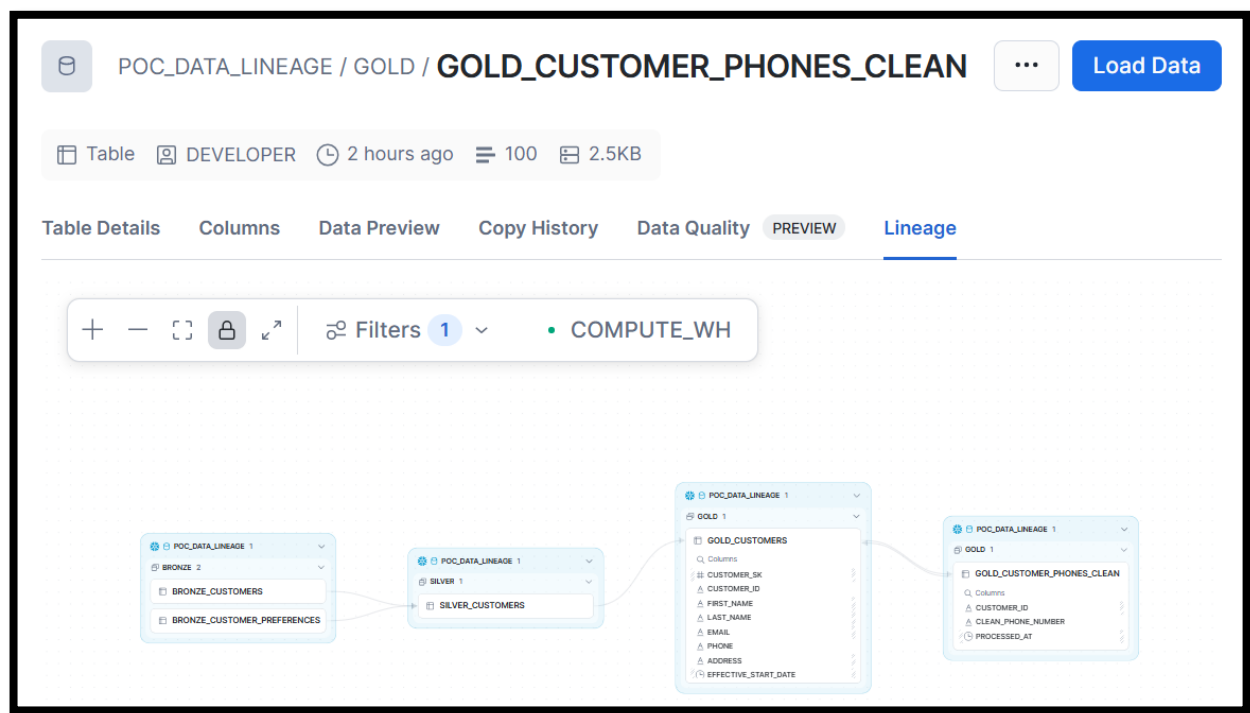
Observation 3: Column level lineage is successfully captured.



Scenario 3: Data Load by Stored procedure written using Javascript

Description: Table GOLD. GOLD_CUSTOMER_PHONES_CLEAN is created using the stored procedure GOLD.SP_GOLD_STANDARDIZE_PHONES using GOLD.GOLD_CUSTOMERS table. GOLD.GOLD_CUSTOMERS table is updated back with standardized Phone numbers from new table GOLD. GOLD_CUSTOMER_PHONES_CLEAN.

Observation 1: Table level lineage successfully captured.



Observation 2: The stored procedure name and query used to load the SILVER.SILVER_CUSTOMERS table is seen.

POC_DATA_LINEAGE / GOLD /

GOLD_CUSTOMER_PHONES_CLEAN

...

Load Data

Table

DEVELOPER

2 hours ago

100

2.5KB

Table Details

Columns

Data Preview

Copy History

Data Quality

PREVIEW

Lineage

+ - [] 🔒 ↗️

Filters 1

C

Diagram showing lineage from GOLD_CUSTOMERS to GOLD_CUSTOMER_PHONES_CLEAN.

GOLD_CUSTOMERS to GOLD_CUSTOMER...

```
CREATE OR REPLACE TABLE  
GOLD.GOLD_CUSTOMER_PHONES_CLEAN AS  
SELECT  
    CUSTOMER_ID,  
    REGEXP_REPLACE(PHONE,  
        '[^0-9]', '') AS CLEAN_PHONE_NUMBER,  
    CURRENT_TIMESTAMP() AS  
    PROCESSED_AT  
FROM GOLD.GOLD_CUSTOMERS
```

Stored Procedure:

SP_GOLD_STANDARDIZE_PHONES()

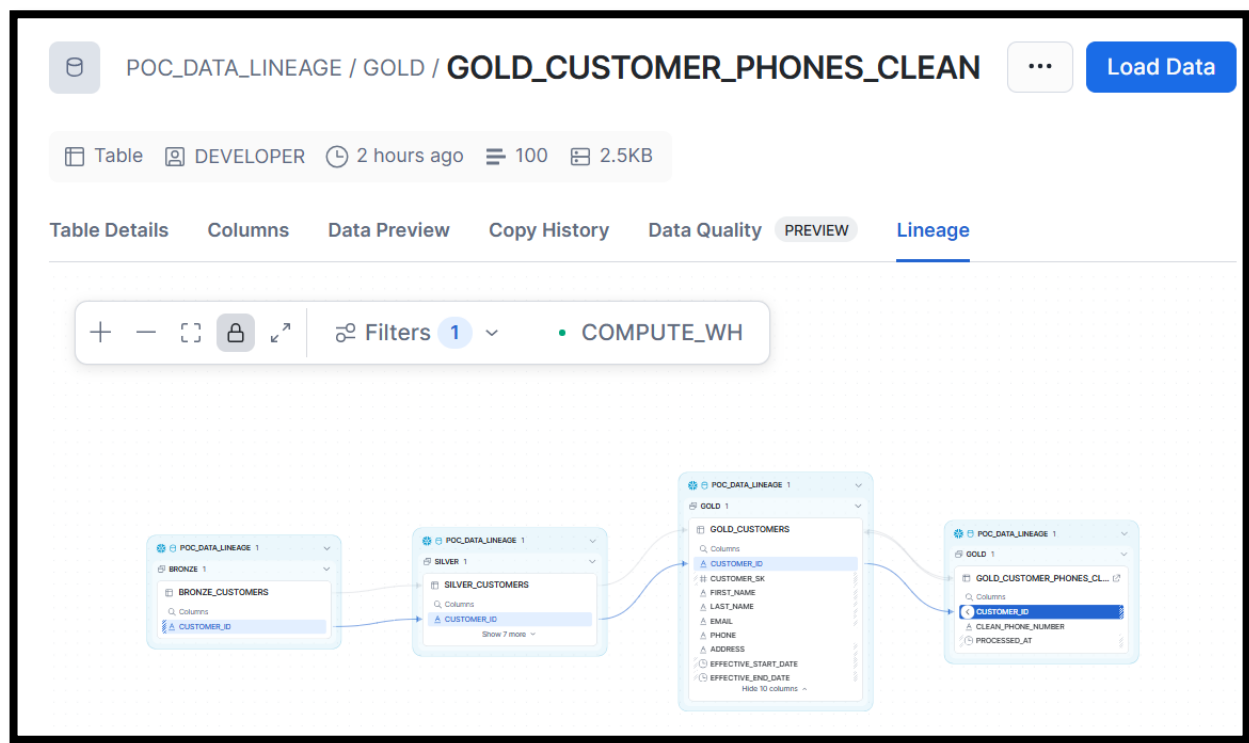
Direct

SP_GOLD_STANDARDIZE_PH

Root

—

Observation 3: Column level lineage successfully captured.



Observation 4: The data lineage for the query that updates the GOLD.GOLD_CUSTOMERS table has also been successfully captured. Query and stored procedure name is also seen.

POC_DATA_LINEAGE / GOLD / **GOLD_CUSTOMER_PHONES_CLEAN**

Table DEVELOPER 2 hours ago 100 2.5KB

Table Details Columns Data Preview Copy History Data Quality PREVIEW **Lineage**

POC_DATA_LINEAGE 1

GOLD 1

GOLD_CUSTOMERS

CUSTOMER_SK

CUSTOMER_ID

FIRST_NAME

LAST_NAME

EMAIL

PHONE

ADDRESS

EFFECTIVE_START_DATE

POC_DATA_LINEAGE 1

GOLD 1

GOLD_CUSTOMER_PHONES_CLEAN

CUSTOMER_ID

CLEAN_PHONE_NUMBER

PROCESSED_AT

GOLD_CUSTOMER_PHONES_CLEAN to G...

UPDATE GOLD.GOLD_CUSTOMERS

AS TGT

SET PHONE =

SRC.CLEAN_PHONE_NUMBER

FROM

GOLD.GOLD_CUSTOMER_PHONES_CLEAN AS

SRC

WHERE TGT.CUSTOMER_ID =

SRC.CUSTOMER_ID

Stored Procedure: SP_GOLD_STANDARDIZE_PHONES()

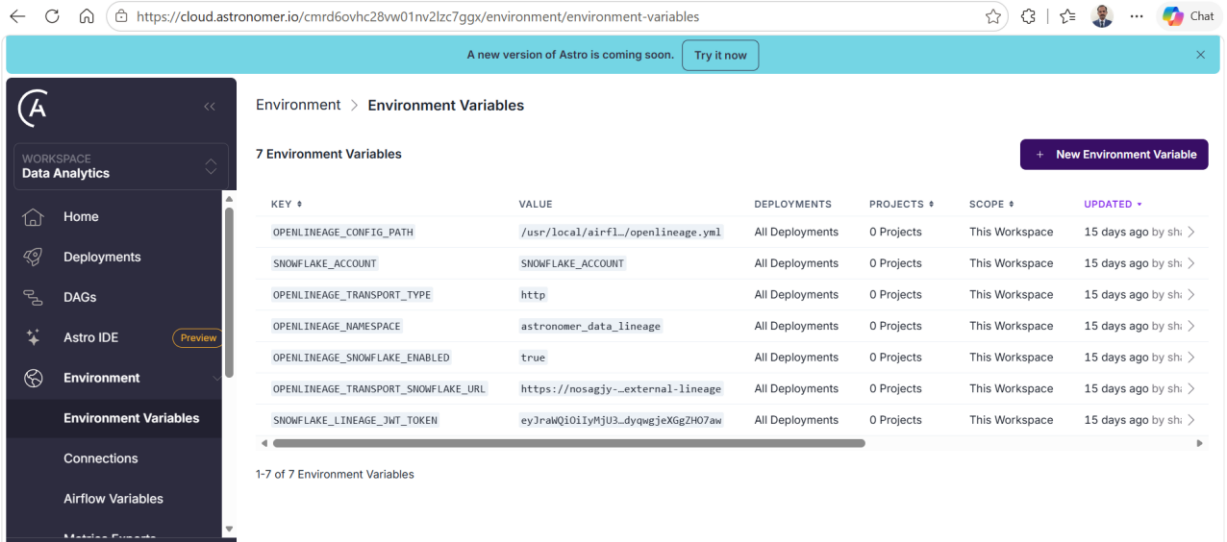
Direct SP_GOLD_STANDARDIZE_PH

Root —

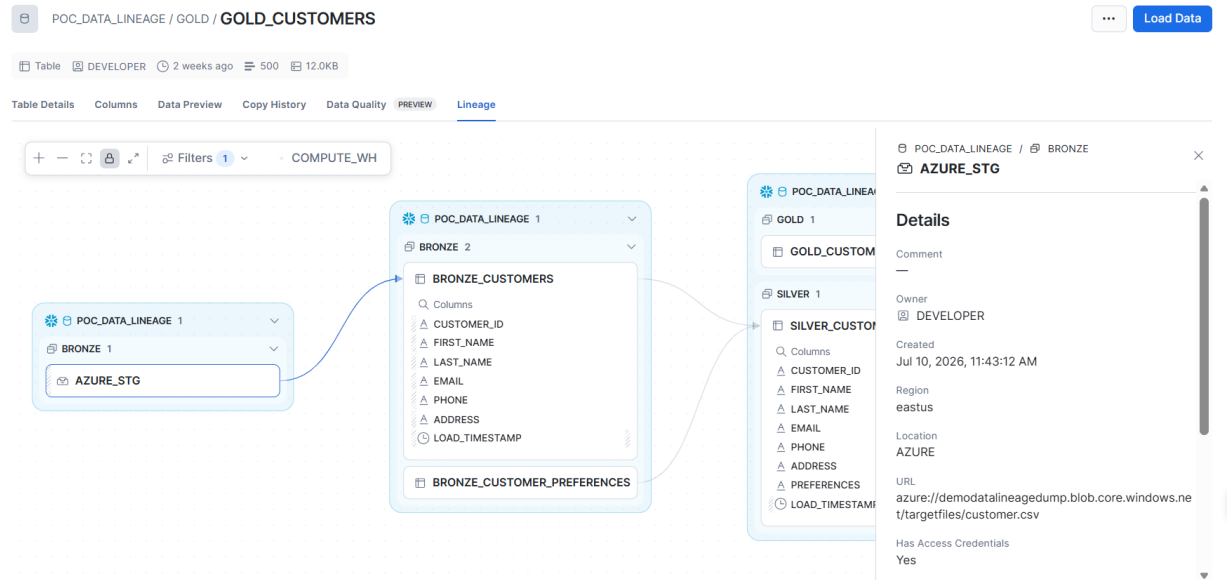
Scenario 4: Airflow External data Lineage for Customer Table

The system tracks data lineage from the Azure External Stage→ Bronze layer, capturing metadata about the customer table's journey through data pipeline.

Data loading to the Bronze layer is orchestrated through an Airflow DAG. To capture lineage from external data sources, we implemented the Snowflake External Lineage API with PAT authentication (credentials managed in Astronomer environment).

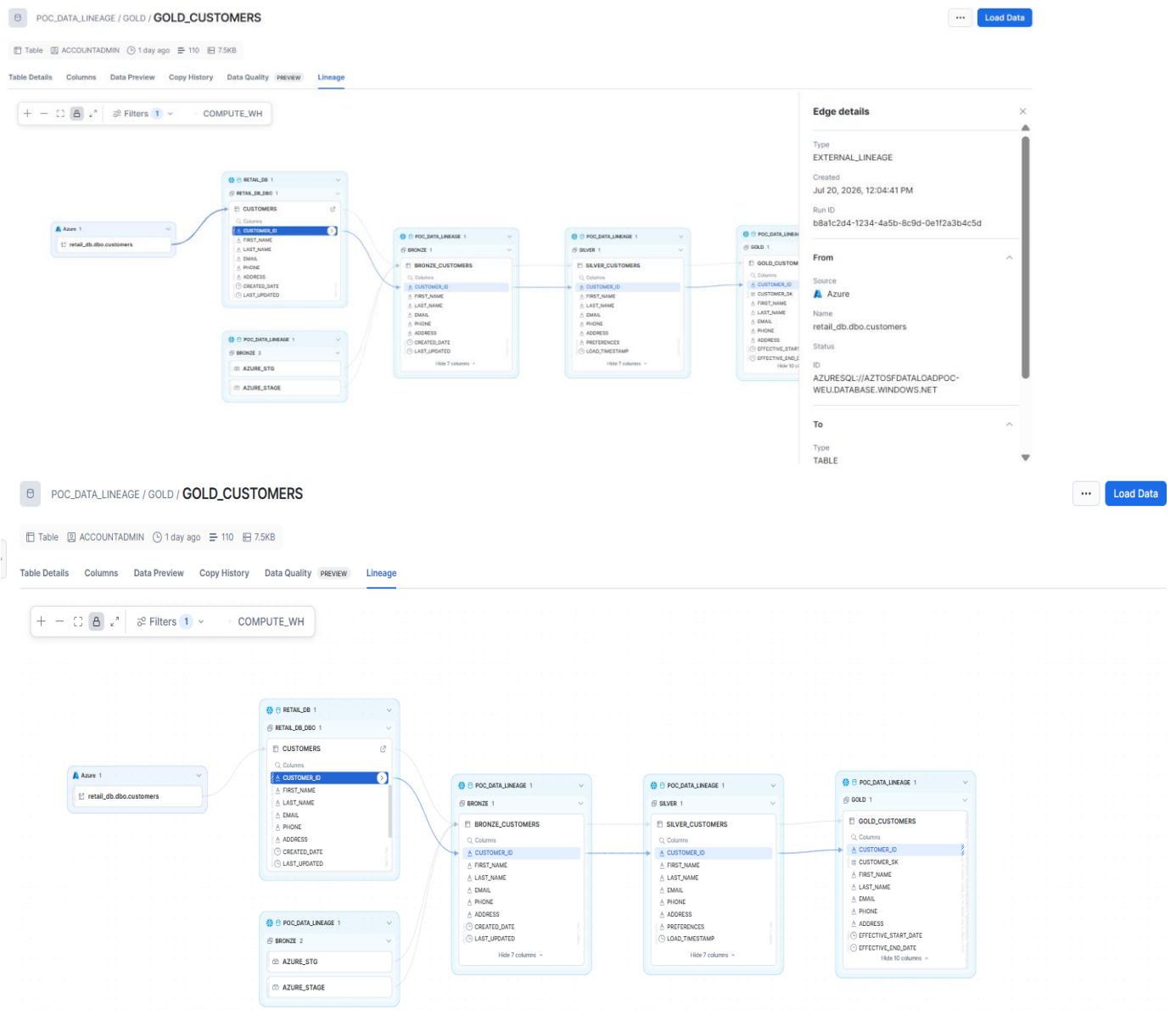


After the DAG loads data to Bronze, lineage metadata is registered to Snowflake via API call. The lineage is then visible in Snowflake's lineage graph. However, it will not show the upstream tool unless it allows to send it through the API payload.



Scenario 5: OpenFlow External Data Lineage for Customer Table

The below will explain the complete upstream and downstream column level Lineage how the table got loaded. It starts from Azure SQL table named retail_db.customers to till GOLD.GOLD_CUSTOMERS.



The below explains the openflow data lineage as external lineage and shows from where the table got loaded.

Scenario 6: External Orchestration Lineage Gap

Snowflake's native lineage (ACCESS_HISTORY, OBJECT_DEPENDENCIES) only captures what happens inside Snowflake. It has no visibility into external orchestration tools like Azure Data Factory (ADF) — pipeline activities, copy activities, data flows, or Nexla — data connectors, transforms, flows.

What's Missing:

Source System (e.g., SQL Server)

| ← ADF / Nexla pipeline (INVISIBLE to Snowflake)

| - extraction logic

| - any in-flight transforms

| - scheduling metadata



RETAIL_DB.RETAIL_DB_DBO.CUSTOMERS (lands in Snowflake)

|

| ← Snowflake lineage starts HERE



BRONZE → SILVER → GOLD

Snowflake's ACCESS_HISTORY picks up lineage starting from the first SQL statement that reads/writes within Snowflake. Everything upstream of that landing zone is a black box.

Current Options to Bridge This Gap

Approach	How It Works	Limitation
OpenLineage / Marquez	ADF and Nexla emit OpenLineage events; store in a central catalog (e.g., Marquez, DataHub, Atlan)	Requires separate lineage catalog outside Snowflake
Custom metadata tables	Write ADF/Nexla run metadata into a Snowflake table (pipeline name, source, destination, run_id, timestamps)	Manual, not integrated with Snowflake lineage UI
Tags on landing tables	ALTER TABLE SET TAG SOURCE_SYSTEM='SQL Server', ETL_TOOL='ADF', PIPELINE_NAME='CustomerSync'	Static metadata only, no run-level lineage
Third-party catalogs	Atlan, Alation, Collibra, Select Star — connect to both ADF/Nexla APIs and Snowflake to stitch end-to-end lineage	Additional cost and tooling

Snowflake Partner Connect	Some partners (e.g., Monte Carlo, Atlan) integrate directly	Vendor-dependent
---------------------------	---	------------------

Scenario 7: Data-Level Lineage for Transformed Columns using CoCo

Description: This scenario tracks data-level lineage for CUSTOMER_ID CUST0107 through a 3-layer medallion architecture using CoCo in snowflake

SOURCE (External)
RETAIL_DB.RETAIL_DB_DBO.CUSTOMERS
Original system of record

SP_BRONZE_CUSTOMERS()
MERGE (incremental by _SNOWFLAKE_UPDATED_AT)



BRONZE.BRONZE_CUSTOMERS
CUST0107 Jason Ryan
jason.ryan229@hotmail.com 3465787133
839 Adrian Station, East Carloston, VI 43810
CREATED: 2026-07-20 01:22:48

SP_SILVER_CUSTOMERS()
MERGE (incremental by LAST_UPDATED)



SILVER.SILVER_CUSTOMERS
CUST0107 Jason Ryan
jason.ryan229@hotmail.com 3465787133
839 Adrian Station, East Carloston, VI 43810
LOAD_TIMESTAMP: 2026-07-20 01:24:04

```
| SP_LOAD_GOLD_CUSTOMERS()  
  
| SCD Type 2 (INSERT new current + expire old)
```



GOLD.GOLD_CUSTOMERS
SK=1 CUST0107 Jason Ryan
jason.ryan229@hotmail.com 3465787133
839 Adrian Station, East Carloston, VI 43810
EFFECTIVE_START: 2026-07-20 01:27:46
IS_CURRENT: TRUE
FULL_NAME: JasonRyan (derived: FIRST+LAST)

Column-Level Lineage:

Gold Column	Source (Silver)	Origin (Bronze)	Transformation
CUSTOMER_ID	CUSTOMER_ID	CUSTOMER_ID	Pass-through
FIRST_NAME	FIRST_NAME	FIRST_NAME	Pass-through
LAST_NAME	LAST_NAME	LAST_NAME	Pass-through
EMAIL	EMAIL	EMAIL	Pass-through
PHONE	PHONE	PHONE	Pass-through
ADDRESS	ADDRESS	ADDRESS	Pass-through
FULL_NAME	FIRST_NAME + LAST_NAME	FIRST_NAME + LAST_NAME	Concatenation

EFFECTIVE_START_DATE	—	—	Generated (CURRENT_TIMESTAMP)
EFFECTIVE_END_DATE	—	—	Generated (NULL or CURRENT_TIMESTAMP)
IS_CURRENT	—	—	Generated (TRUE/FALSE)
CUSTOMER_SK	—	—	Generated (sequence)

Key Observations:

- Source system: RETAIL_DB.RETAIL_DB_DBO.CUSTOMERS (likely an external connector/replicated database)
- 3-layer medallion architecture: Bronze (raw) → Silver (cleansed) → Gold (business-ready SCD2)
- SCD Type 2 applied at the Gold layer — tracks historical changes with effective dates
- FULL_NAME is derived by concatenating FIRST_NAME and LAST_NAME (no separator)
- Single record exists for CUST0107 (no historical versions yet — first loaded 2026-07-20)

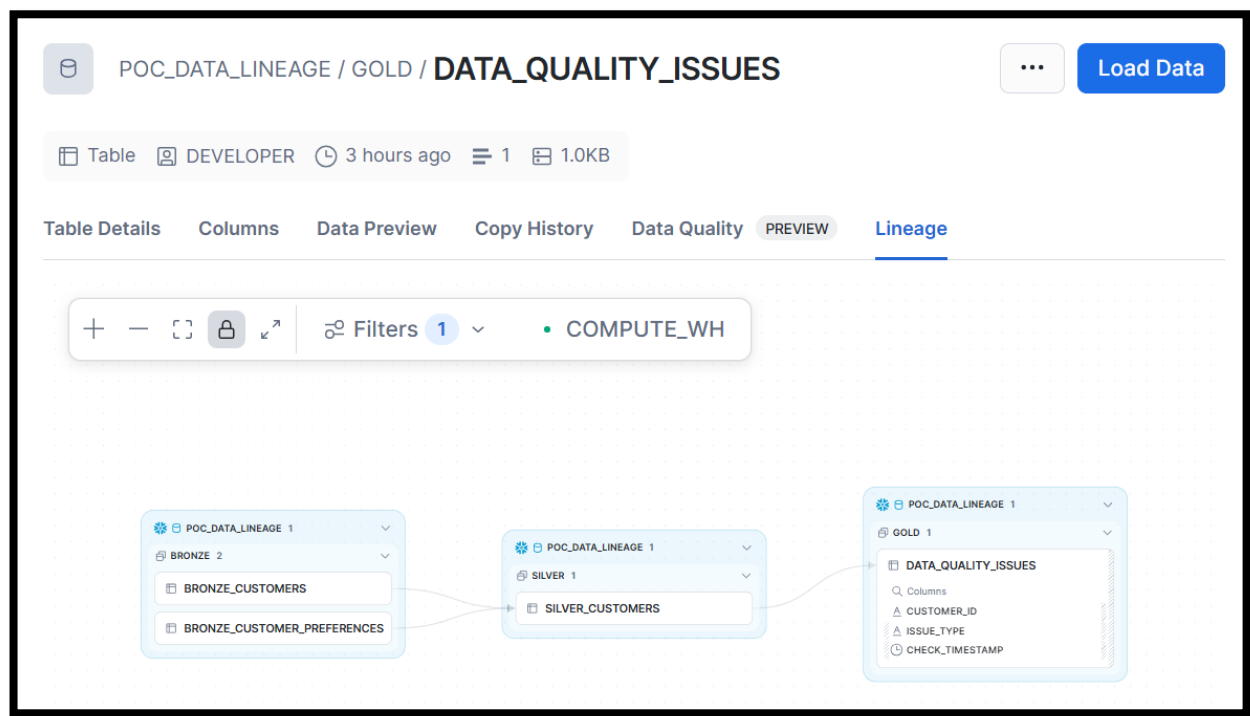
Limitations:

Snowflake does not natively support row/value-level lineage inside procedural logic, so any solution requires instrumenting your stored procedures to emit that information explicitly

Scenario 8: Data Load by Snowpark Stored procedure

Description: Table GOLD. DATA_QUALITY_ISSUES is loaded from SILVER.SILVER_CUSTOMERS table using Snowpark stored procedure GOLD.SP_GOLD_QUALITY_ASSURANCE_AUDIT.

Observation 1: Table level lineage successfully captured.



Observation 2: The stored procedure name and query used to load the `SILVER.SILVER_CUSTOMERS` table is seen.

POC_DATA_LINEAGE / GOLD / DATA_QUALITY_ISSUES

Table

DEVELOPER

3 hours ago

1

1.0KB

Table Details

Columns

Data Preview

Copy History

Data Quality

PREVIEW

Lineage

POC_DATA_LINEAGE 1

SILVER 1

SILVER_CUSTOMERS

POC_DATA_LINEAGE 1

GOLD 1

DATA_QUALITY_ISSUES

CUSTOMER_ID

ISSUE_TYPE

CHECK_TIMESTAMP

SILVER_CUSTOMERS to DATA_QUALITY_I...

SQL Query

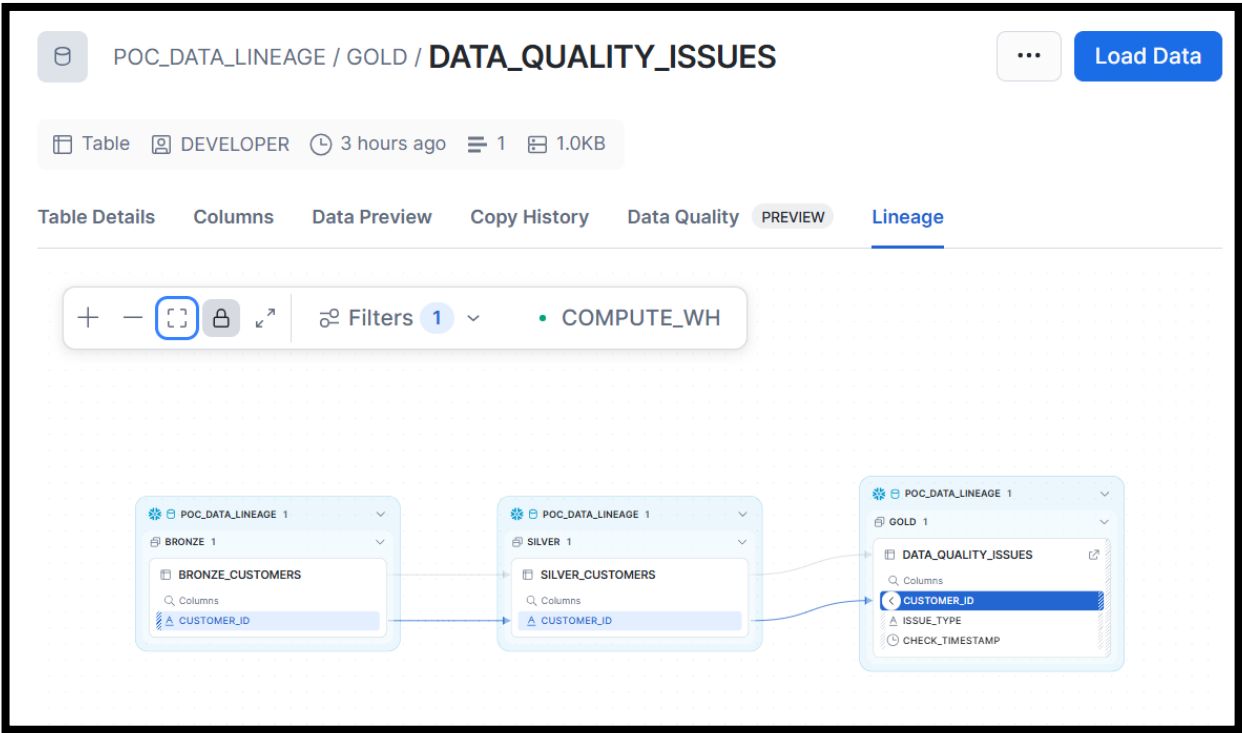
```
INSERT INTO
GOLD.DATA_QUALITY_ISSUES SELECT
  CAST ("CUSTOMER_ID" AS STRING),
  'Missing or Malformed Email' AS
"ISSUE_TYPE",
  current_timestamp() AS
"CHECK_TIMESTAMP"
FROM SILVER.SILVER_CUSTOMERS
WHERE
  (NOT "EMAIL" LIKE '%@%' OR
```

Stored Procedures

Direct

SP_GOLD_QUALITY_ASSURAN

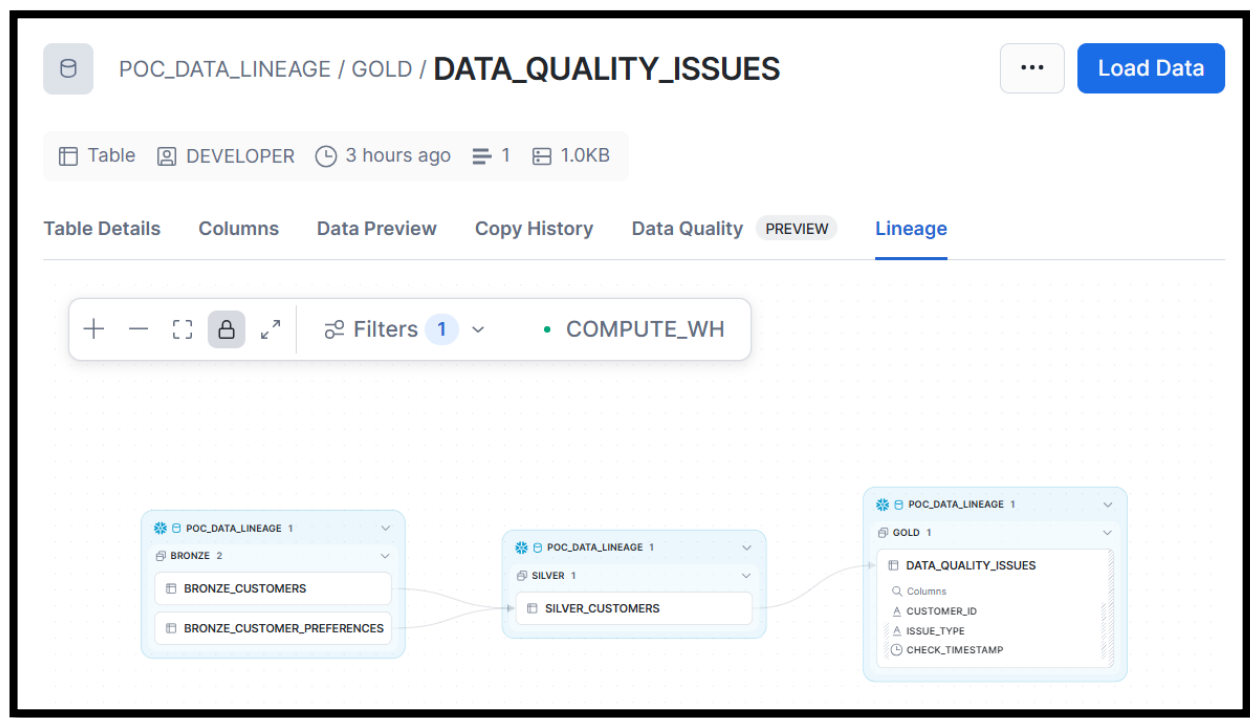
Observation 3: Column level lineage successfully captured.



Scenario 9: Use python library function for calculations/transformations

Description: Table GOLD. DATA_QUALITY_ISSUES is loaded from SILVER.SILVER_CUSTOMERS table using Snowpark stored procedure GOLD.SP_GOLD_QUALITY_ASSURANCE_AUDIT.

Observation 1: Table level lineage successfully captured.



Observation 2: The stored procedure name and query used to load the SILVER.SILVER_CUSTOMERS table is seen.

POC_DATA_LINEAGE / GOLD / DATA_QUALITY_ISSUES

Table DEVELOPER 3 hours ago 1 1.0KB

Table DetailsColumnsData PreviewCopy HistoryData QualityPREVIEWLineage

POC_DATA_LINEAGE 1

SILVER 1

SILVER_CUSTOMERS

POC_DATA_LINEAGE 1

GOLD 1

DATA_QUALITY_ISSUES

CUSTOMER_ID

ISSUE_TYPE

CHECK_TIMESTAMP

SILVER_CUSTOMERS to DATA_QUALITY_I...

SQL Query

```
INSERT INTO
GOLD.DATA_QUALITY_ISSUES SELECT
  CAST ("CUSTOMER_ID" AS STRING),
  'Missing or Malformed Email' AS
"ISSUE_TYPE",
  current_timestamp() AS
"CHECK_TIMESTAMP"
FROM SILVER.SILVER_CUSTOMERS
WHERE
  (NOT "EMAIL" LIKE '%@%' OR
```

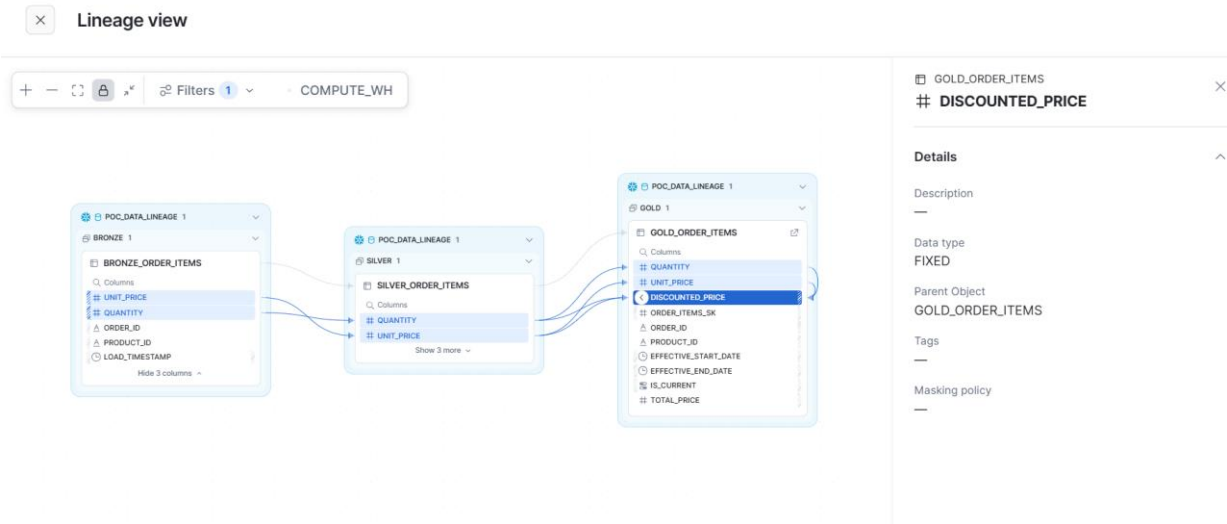
Stored Procedures

DirectSP_GOLD_QUALITY_ASSURAN

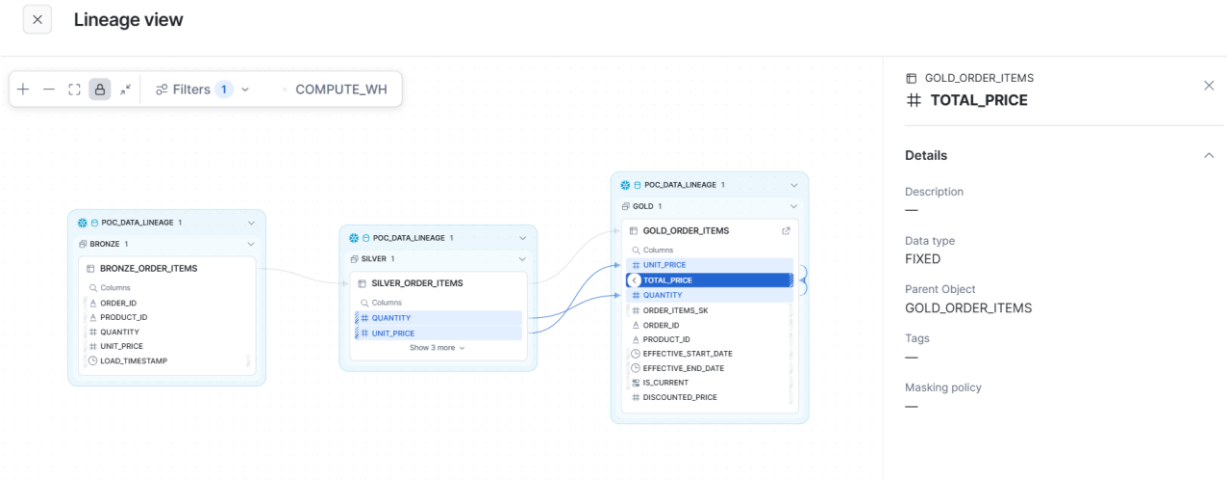
Observation 3: Column level lineage successfully captured.

Scenario 10: Use python library function for calculations/transformations to update a column.

Create a python language stored procedure to update a column from SILVER.SILVER_ORDER_ITEMS.Unit_Price and Quantity. Using these columns, a new column DISCOUNTED_PRICE has been added in GOLD.GOLD_ORDER_ITEMS and updated using python stored procedure. However, it will not show the SP or Logic how it got updated

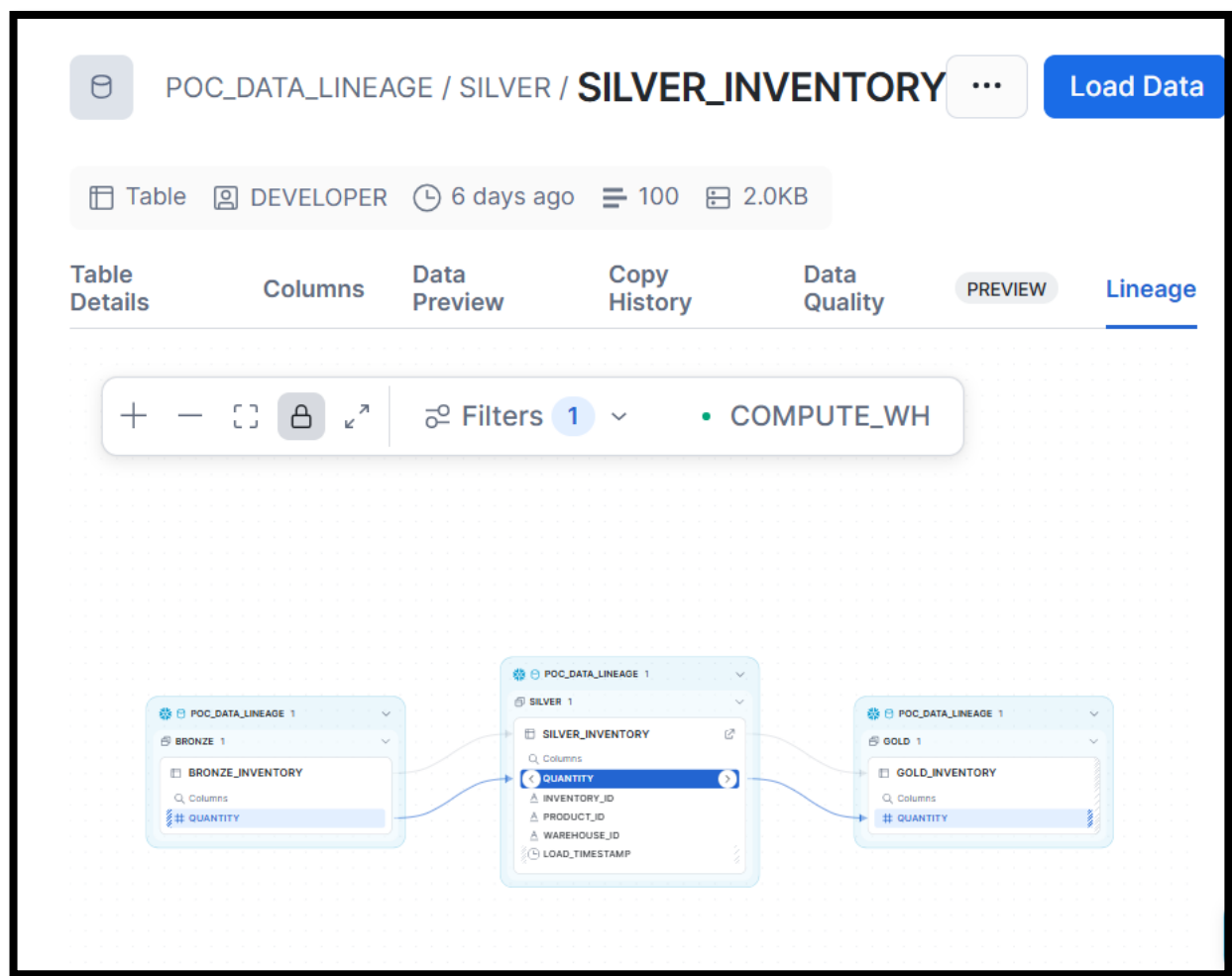


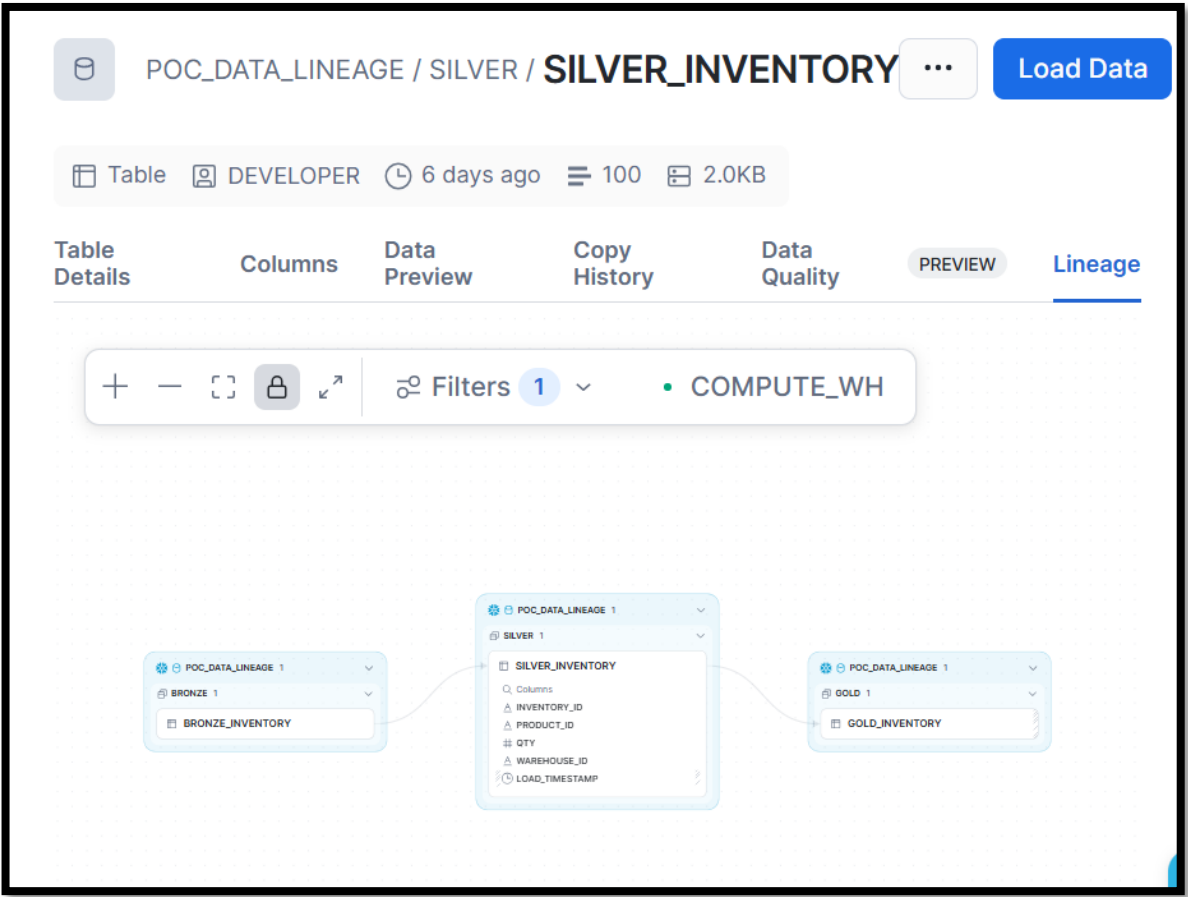
Create a python language stored procedure to update a column from GOLD.GOLD_ORDER_ITEMS.Unit_Price and Quantity. Using these columns, a new column TOTAL_PRICE has been added in GOLD.GOLD_ORDER_ITEMS and updated using python stored procedure. However, it will not show the SP or Logic how it got updated



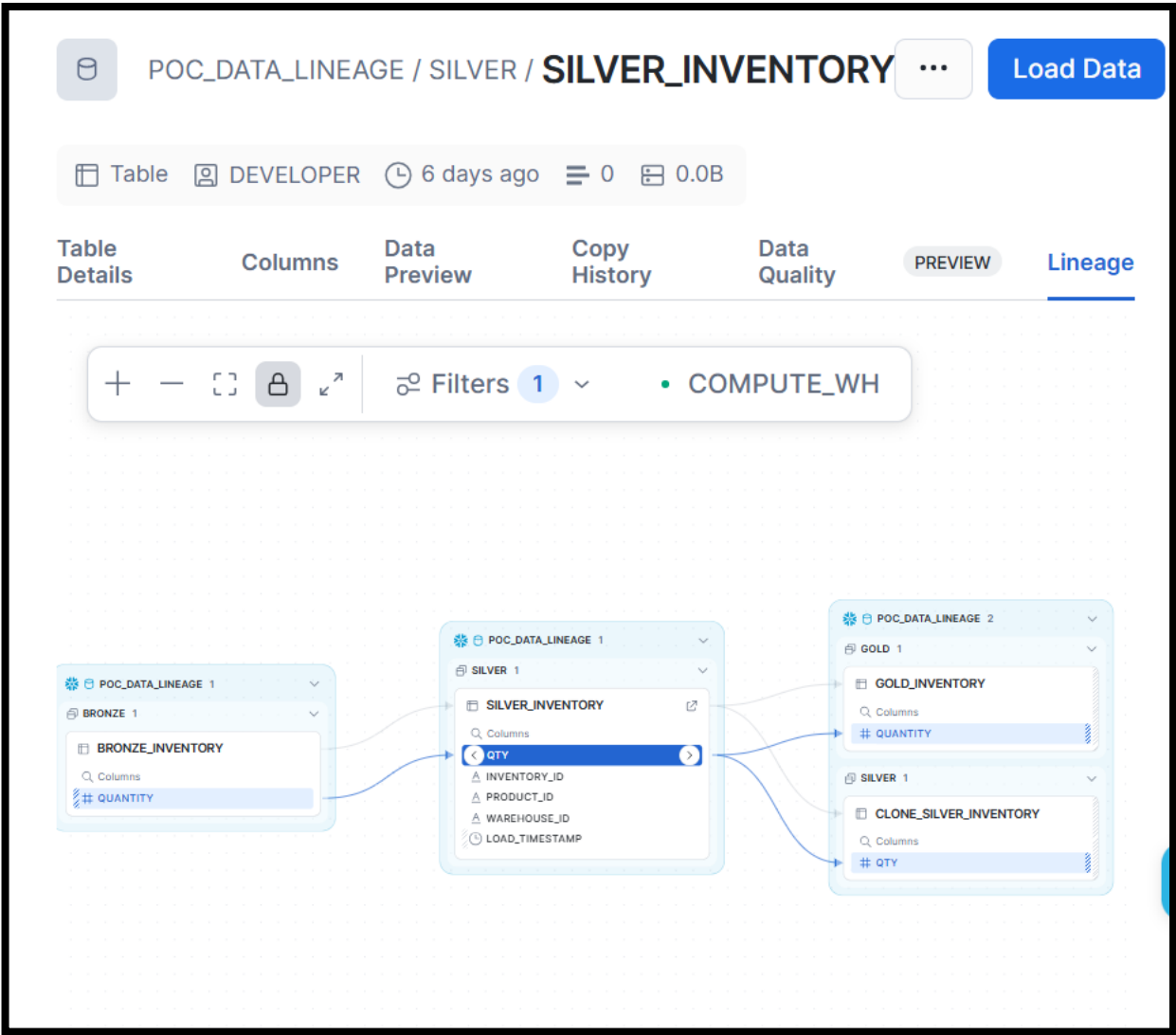
Scenario 11: Rename a column in a silver table & check if the lineage shows the updated name. Do it without the data load.

There is column QUANTITY in SILVER.SILVER_INVENTROY had been changed to QTY where it has shown the column lineage even after renaming the column.

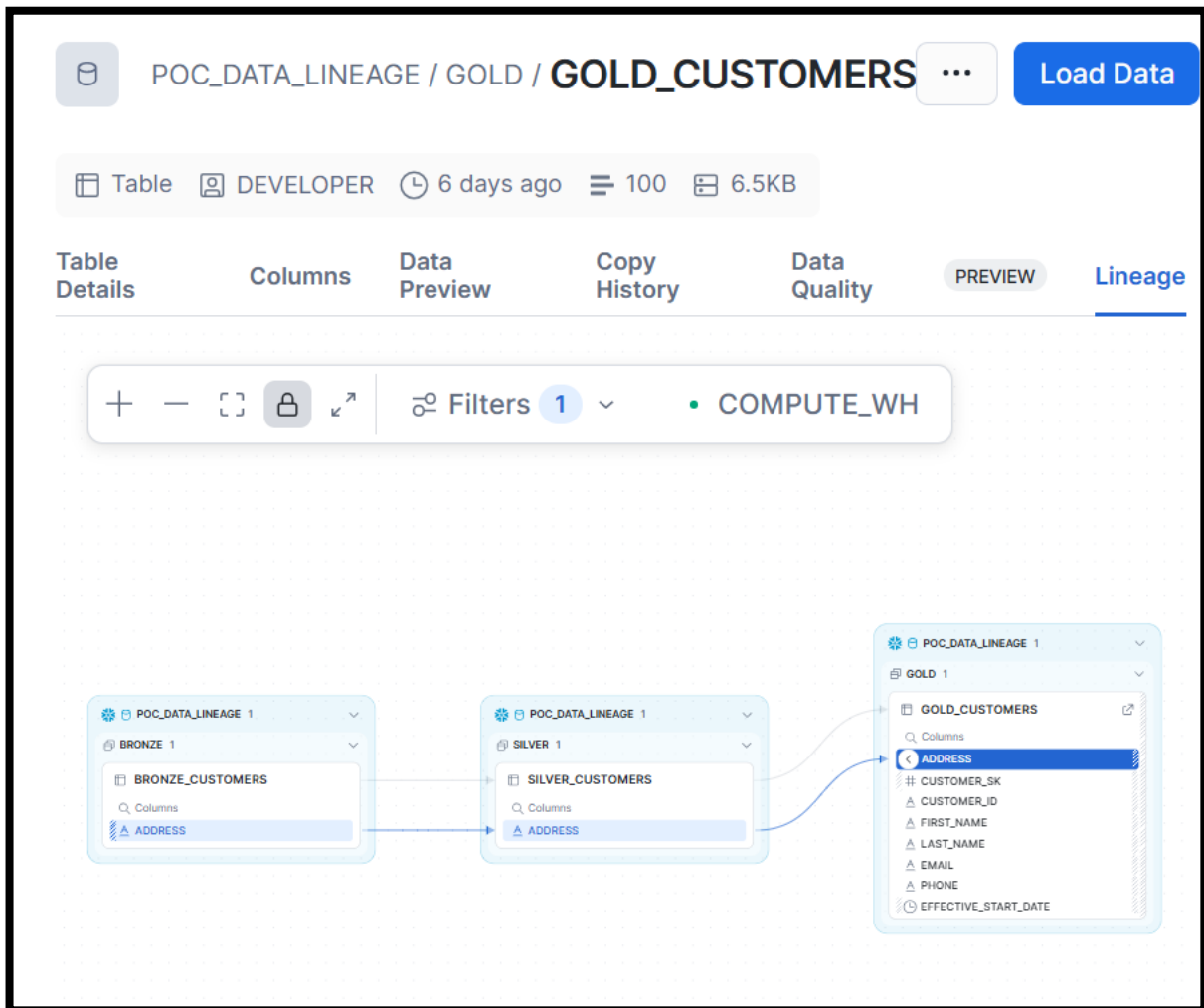




The below screen shows the column lineage in the GOLD.GOLD_INVENTORY after renaming SILVER.SILVER_INVENTORY.QTY column



Scenario 12: Delete 1 column from the Bronze table & generate the lineage from the corresponding gold table without the data load.



POC_DATA_LINEAGE / GOLD / **GOLD_CUSTOMERS**

Load Data

Table

DEVELOPER

6 days ago

100

6.5KB

Table Details

Columns

Data Preview

Copy History

Data Quality

PREVIEW

Lineage

Filters 1

COMPUTE_WH

POC_DATA_LINEAGE 1

BRONZE 1

BRONZE_CUSTOMERS

Columns

CUSTOMER_ID

FIRST_NAME

LAST_NAME

EMAIL

PHONE

CREATED_DATE

LAST_UPDATED

POC_DATA_LINEAGE 1

SILVER 1

SILVER_CUSTOMERS

Columns

ADDRESS

CUSTOMER_ID

FIRST_NAME

LAST_NAME

EMAIL

PHONE

LOAD_TIMESTAMP

POC_DATA_LINEAGE 1

GOLD 1

GOLD_CUSTOMERS

Columns

ADDRESS

CUSTOMER_SK

CUSTOMER_ID

FIRST_NAME

LAST_NAME

EMAIL

PHONE

EFFECTIVE_START_DATE

EFFECTIVE_END_DATE

Scenario 13: Drop a bronze table & generate the lineage without data load.

POC_DATA_LINEAGE / GOLD / **GOLD_CUSTOMERS**

Load Data

Table

DEVELOPER

6 days ago

100

6.5KB

Table Details

Columns

Data Preview

Copy History

Data Quality

PREVIEW

Lineage

Filters 1

COMPUTE_WH

POC_DATA_LINEAGE 1

SILVER 1

SILVER_CUSTOMERS

POC_DATA_LINEAGE 1

GOLD 1

GOLD_CUSTOMERS

Columns

CUSTOMER_SK

CUSTOMER_ID

FIRST_NAME

LAST_NAME

EMAIL

PHONE

ADDRESS

EFFECTIVE_START_DATE

Scenario 14: GET LINEAGE FUNCTION

GET_LINEAGE Function will help to check the table and column level lineage at the query level. The below is query for checking the column level lineage for BRONZE.BRONZE_CUSTOMERS.CUSTOMER_ID' for its upstream and downstream

```
SELECT DISTANCE, SOURCE_OBJECT_SCHEMA, SOURCE_OBJECT_NAME, SOURCE_COLUMN_NAME,
TARGET_OBJECT_SCHEMA, TARGET_OBJECT_NAME, TARGET_COLUMN_NAME
FROM TABLE(SNOWFLAKE.CORE.GET_LINEAGE(
'POC_DATA_LINEAGE.BRONZE.BRONZE_CUSTOMERS.CUSTOMER_ID',
'COLUMN', 'DOWNSTREAM', 5))
ORDER BY DISTANCE;
```

Results (just now)

TableChartPivot

5 rows970ms

	DISTANCE	SOURCE_OBJECT_SCHEMA	SOURCE_OBJECT_NAME	SOURCE_COLUMN_NAME	TARGET_OBJECT_SCHEMA	TARGET_OBJECT_NAME	TARGET_COLUMN_NAME
1	1	BRONZE	BRONZE_CUSTOMERS	CUSTOMER_ID	SILVER	SILVER_CUSTOMERS	CUSTOMER_ID
2	2	SILVER	SILVER_CUSTOMERS	CUSTOMER_ID	GOLD	VW_GOLD_CUSTOEMRS	CUSTOMER_ID
3	2	SILVER	SILVER_CUSTOMERS	CUSTOMER_ID	GOLD	GOLD_CUSTOMERS	CUSTOMER_ID
4	2	SILVER	SILVER_CUSTOMERS	CUSTOMER_ID	GOLD	DATA_QUALITY_ISSUES	CUSTOMER_ID
5	3	GOLD	GOLD_CUSTOMERS	CUSTOMER_ID	GOLD	GOLD_CUSTOMER_PHONES_CL	CUSTOMER_ID

The below is query for checking the table level lineage for BRONZE.BRONZE_CUSTOMERS

```
SELECT DISTANCE, SOURCE_OBJECT_DOMAIN, SOURCE_OBJECT_SCHEMA, SOURCE_OBJECT_NAME,
TARGET_OBJECT_SCHEMA, TARGET_OBJECT_NAME FROM
TABLE(SNOWFLAKE.CORE.GET_LINEAGE('POC_DATA_LINEAGE.BRONZE.BRONZE_CUSTOMERS','TABLE',
'DOWNSTREAM', 5)) ORDER BY DISTANCE;
```

Results (just now)

TableChartPivot

7 rows1.4s

	# DISTANCE	SOURCE_OBJECT_DOMAIN	SOURCE_OBJECT_SCHEMA	SOURCE_OBJECT_NAME	TARGET_OBJECT_SCHEMA	TARGET_OBJECT_NAME
1	1	TABLE	BRONZE	BRONZE_CUSTOMERS	SILVER	SILVER_CUSTOMERS
2	2	TABLE	SILVER	SILVER_CUSTOMERS	GOLD	VW_GOLD_CUSTOEMRS
3	2	TABLE	SILVER	SILVER_CUSTOMERS	GOLD	DATA_QUALITY_ISSUES
4	2	TABLE	SILVER	SILVER_CUSTOMERS	GOLD	GOLD_CUSTOMERS
5	3	TABLE	GOLD	GOLD_CUSTOMERS	GOLD	GOLD_CUSTOMER_PHONES_CLEAN
6	4	TABLE	GOLD	GOLD_CUSTOMER_PHONES_CLEAN	GOLD	GOLD_CUSTOMERS
7	5	TABLE	GOLD	GOLD_CUSTOMERS	GOLD	GOLD_CUSTOMER_PHONES_CLE